TOP / MEMVID EDITION

Spring Boot + FastAPI 하이브리드 구조

업무 통제는 Spring, 사건 메모리·검색은 Python + memvid

쉽게 말하면

Spring이 누가 어떤 사건을 처리할 수 있는지 확정하고, Python은 승인된 사건 파일에서 근거를 찾아 보고서를 만듭니다. 외부 검색 DB 중심 구성을 사건별 memvid 메모리로 바꾼 개정 설계입니다.

읽는 순서	이번 문서의 범위
TOP · 큰 지도	두 언어의 역할, 전체 구조, 요청 한 건의 순서
MIDDLE · 통제와 내부 흐름	내부 토큰, 스크리닝, 가드, 과금, 이벤트, 사건 격리, 개인정보 파기
BOTTOM · 개선과 개발	동시성·속도, 단계적 교체, 에이전트 병렬 개발, 모듈·검증 지도

원본의 13개 주제를 유지하고 도식을 가로형 벡터로 다시 그렸습니다. 원문에 나온 코드 경로·수치·테스트 이력은 당시 설명이며, 이 개정판의 신규 기능이 구현되었다는 뜻은 아닙니다.

TOP / NAVIGATION

학습 목차

각 항목을 누르거나 PDF 책갈피로 이동합니다.

01 왜 두 언어로 나누나

02 전체 구조

03 요청 한 건

04 내부 토큰

05 스크리닝 전처리

05 생성·사후 검사

06 Dynamic Guard

07 예산·정산

08 이벤트·퍼널

09 사건별 격리

10 개인정보 파기

11 동시성·속도

12 리팩토링

13 병렬 개발

모듈·검증 지도

근거·변경 내역

01 · 왜 두 언어로 나누나

쉽게 말하면

돈과 사람과 업무 기록은 Spring, 사건 자료의 검색과 모델 실험은 Python에 맡깁니다. 한 언어로도 만들 수 있지만 우리 팀은 변경 주기와 실패 경계를 분리합니다.

맡는 일	Spring Boot · Java 21	FastAPI · Python + memvid
업무	로그인·권한·상담·예약·트랜잭션·감사	사건별 memvid 검색·임베딩·모델·JSON
변경	업무 규칙·권한 일관성을 중심으로 검토	모델·검색 설정을 평가 후 교체
실패 통제	중복 저장·원장 불일치 방지, 정책 승인	시간·반복·출력 상한, 오류와 사용량 반환
비용과 운영	예산 승인·예약·원장·관리자 화면	승인된 모델 실행·실제 사용량 보고

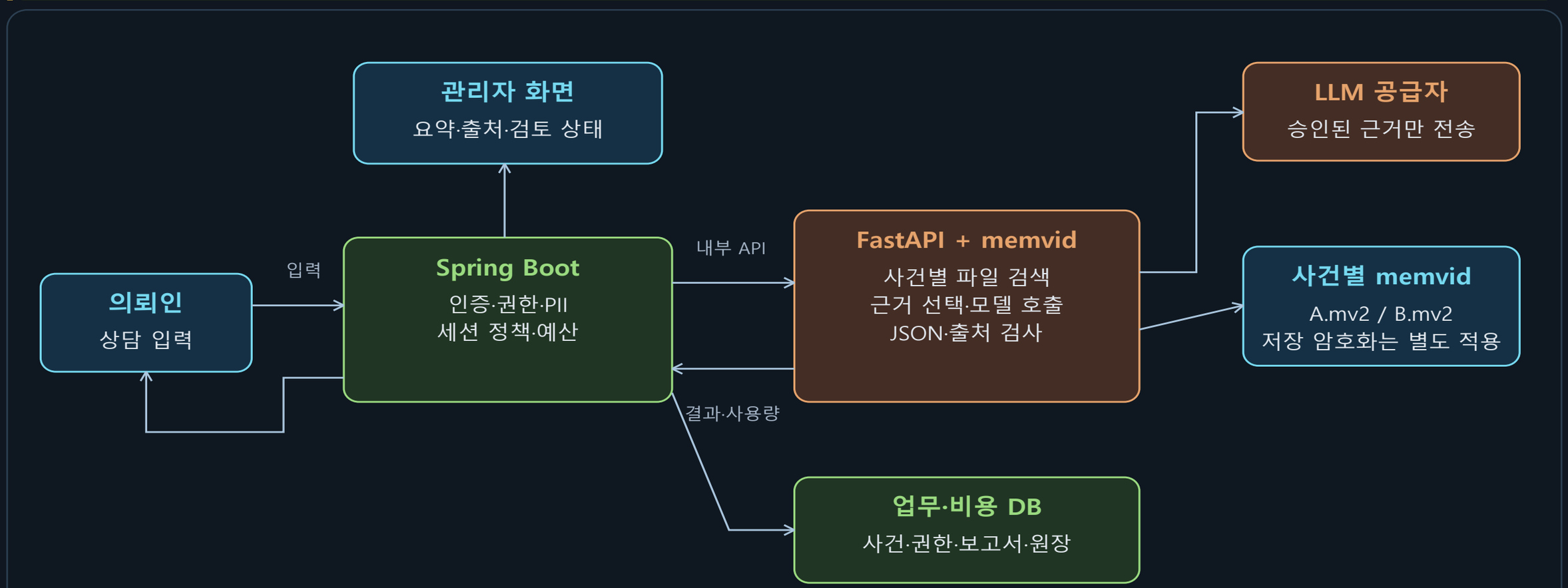
Python에서 틀려도 된다는 뜻은 아닙니다. 개인정보 전송과 유료 호출은 실행 전 통제가 필요합니다. Java 21의 가상 스레드 활성화 여부도 별도로 확인합니다.

TOP / SYSTEM MAP

02 · 전체 구조: 통제와 실행의 경계

쉽게 말하면

Spring에서 권한과 정책을 결정하고, FastAPI에서 같은 범위를 집행합니다. memvid 파일은 검색용 기억이며 업무 DB와 비용 원장을 대신하지 않습니다.



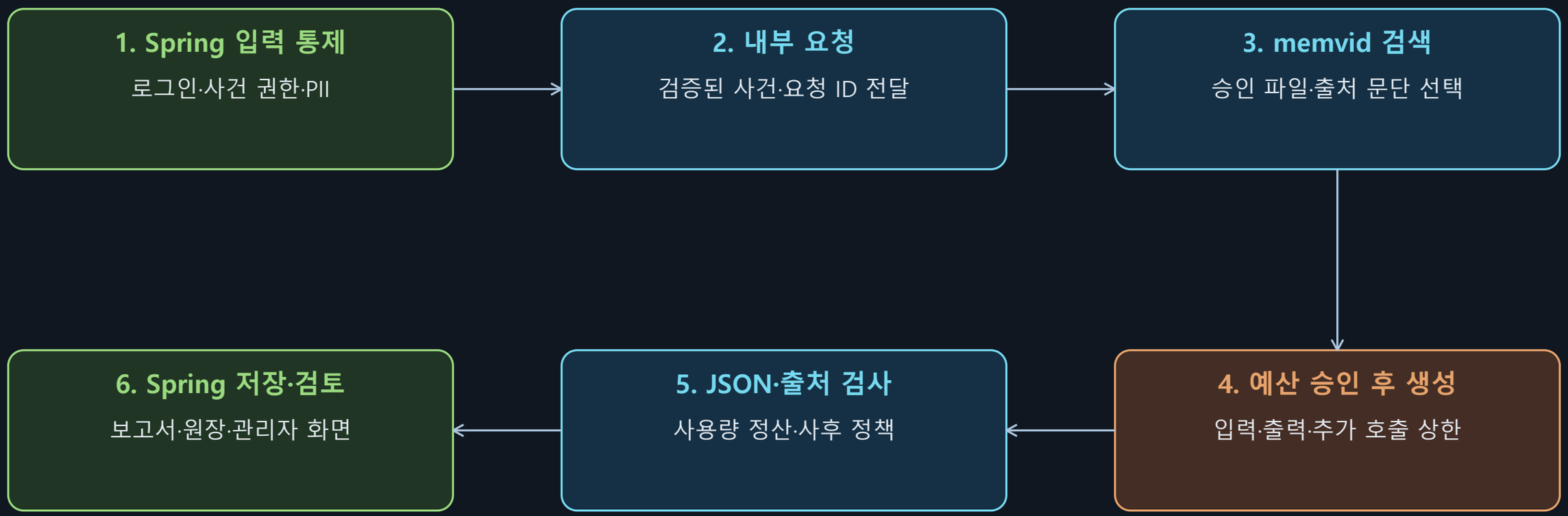
도식의 서버·저장소는 개정 설계입니다. Spring↔Python 내부 인증, 사건 매핑, 예산 승인, 사용량 정산을 API 계약으로 고정합니다.

MIDDLE / REQUEST

03 · 상담 요청 한 건이 지나는 길

쉽게 말하면

입력 통제와 근거 선택을 먼저 끝내고 최종 프롬프트를 기준으로 모델 실행을 승인합니다. 기본 생성 1회를 목표로 하고 추가 호출은 별도 예산에 넣습니다.



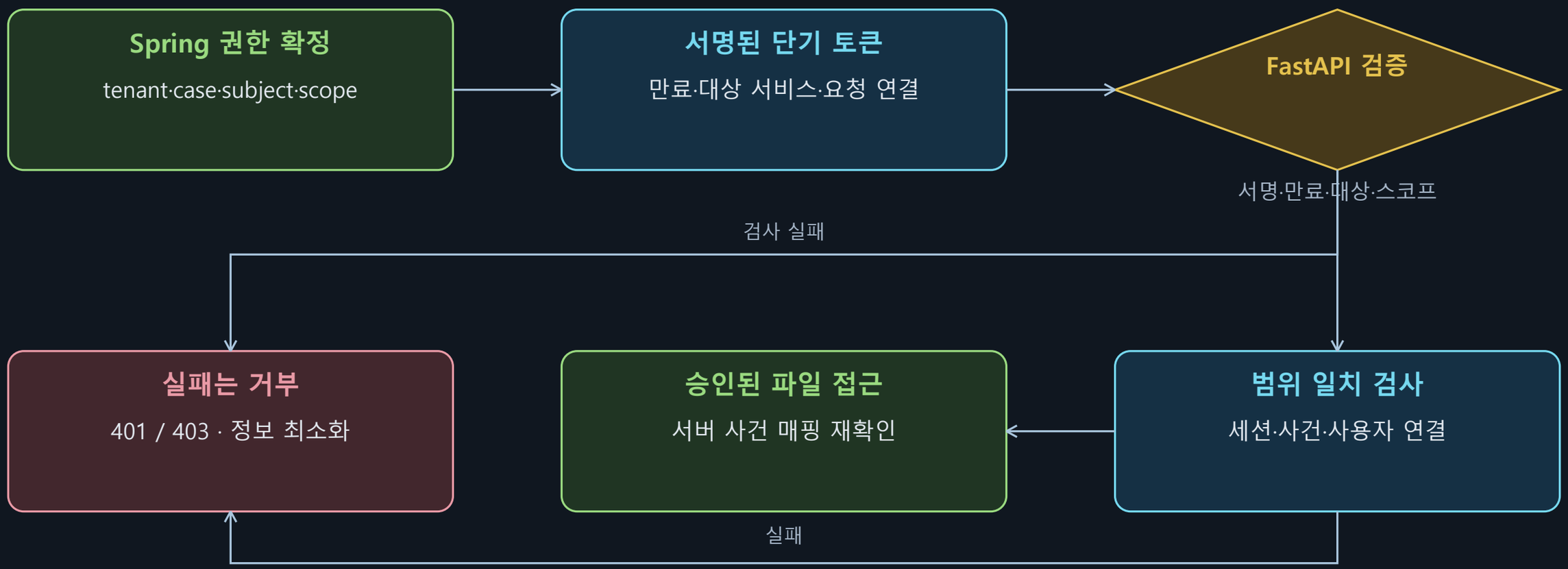
화살표는 위쪽 왼쪽→오른쪽, 아래쪽 오른쪽→왼쪽 순서입니다. 모델 응답 지연 시 타임아웃·재조회·미확정 사용량을 구분합니다.

MIDDLE / IDENTITY

04 · 두 서버의 약속: 내부 토큰

쉽게 말하면

로펌·사건 권한은 검증된 인증 문맥에서 가져옵니다. 요청 본문이 주장하는 tenant_id나 파일 경로로 권한을 새로 만들지 않습니다.



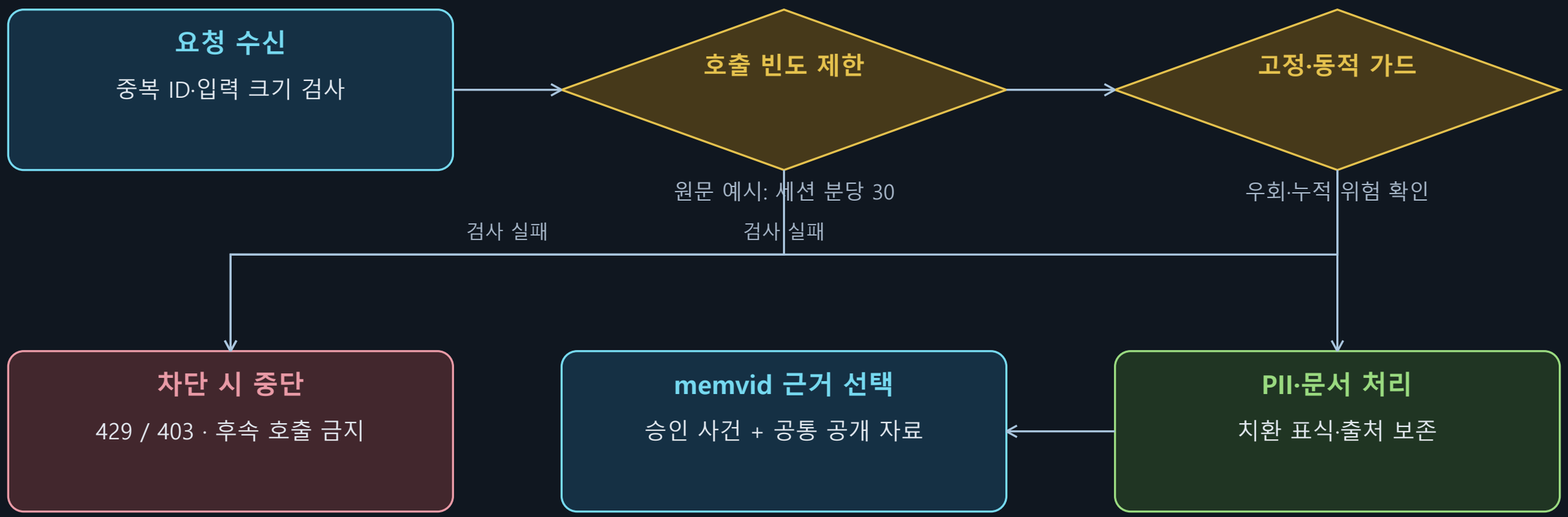
원문은 HMAC 기반 토큰을 설명합니다. HMAC은 암호화가 아니므로 본문은 읽힐 수 있습니다. TLS·비밀 키 분리·키 회전·재전송 정책을 함께 설계합니다.

MIDDLE / INPUT PIPELINE

05-A · 스크리닝: 저비용 검사를 먼저

쉽게 말하면

속도 제한과 입력 검사를 통과한 요청만 검색·유료 단계로 보냅니다. 차단 전에 이미 수행한 OCR·임베딩 등의 비용이 있는지도 따로 기록합니다.



30회는 원문 정책 예시이며 운영 한도는 설정으로 관리합니다. 사건 비공개 파일과 공통 지식 파일은 별도 범위로 열고 결과의 출처도 구분합니다.

MIDDLE / OUTPUT PIPELINE

05-B · 생성 이후: 정산과 사후 검사

쉽게 말하면

출력이 차단되어도 이미 수행한 모델 호출 비용은 남습니다. 출처 검사와 보안 검사는 서로 다른 책임이며 어느 하나가 나머지를 대신하지 않습니다.



원문 복사 50%, 추출 점수 +50·누적 100 등의 수치는 원문의 정책 예시입니다. 검색되지 않은 출처 ID는 거부·재검토하며 문장에 링크만 붙여 사실로 만들지 않습니다.

MIDDLE / GUARD

06 · Dynamic Guard: 단건과 누적

쉽게 말하면

현재 문장과 최근 대화 조각을 함께 검사하되 정상적인 피해 설명을 공격으로 오인하지 않도록 합니다. 규칙 점수는 비용이나 정답 확률이 아닙니다.

검사 단계	원문 설명을 보존한 학습용 정책
정규화	NFKC·보이지 않는 문자 처리. 공백을 모두 없애 정상 문장을 오탐하지 않도록 검토
단건 검사	탈옥 60 / 주입 60 / 비밀 요청 50 / 대량 추출 50 / 과다 길이 35
분산 검사	최근 메시지 5개에서 나누어 보낸 주입 시도 확인
누적과 알림	누적 100 이상을 제한하는 예시. 점수·사유·정책 버전 기록
오탐 검증	피해 진술과 비밀 요구 명령을 구분하고 운영자 검토·해제 절차 마련

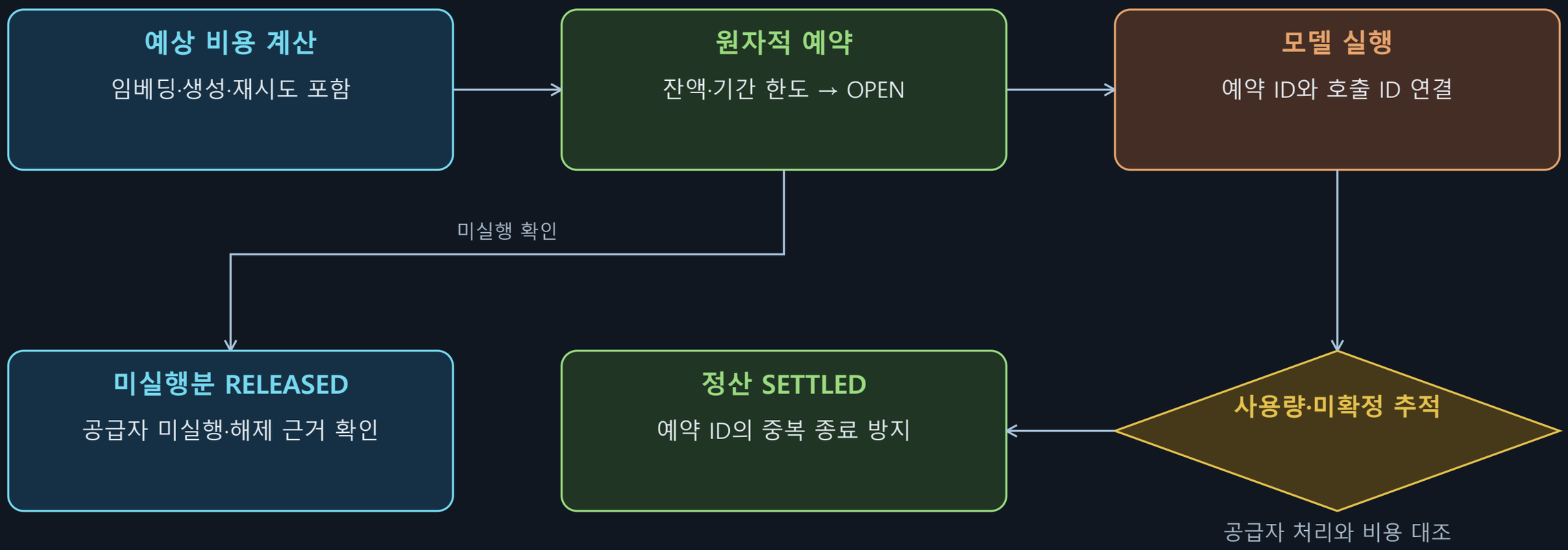
위 수치와 규칙은 원문 사례이며 현재 코드 실행으로 재검증하지 않았습니다. Python의 신규 탐지 이벤트는 인증·중복 방지를 거쳐 Spring 정책 이력과 연결합니다.

MIDDLE / BILLING

07 · 비용: 예약 → 실행 → 정산

쉽게 말하면

사용 가능한 잔액에서는 진행 중 예약도 빼야 합니다. 동일 요청의 완료·오류·재전송이 같은 예약을 두 번 닫지 않도록 상태 전이를 제한합니다.



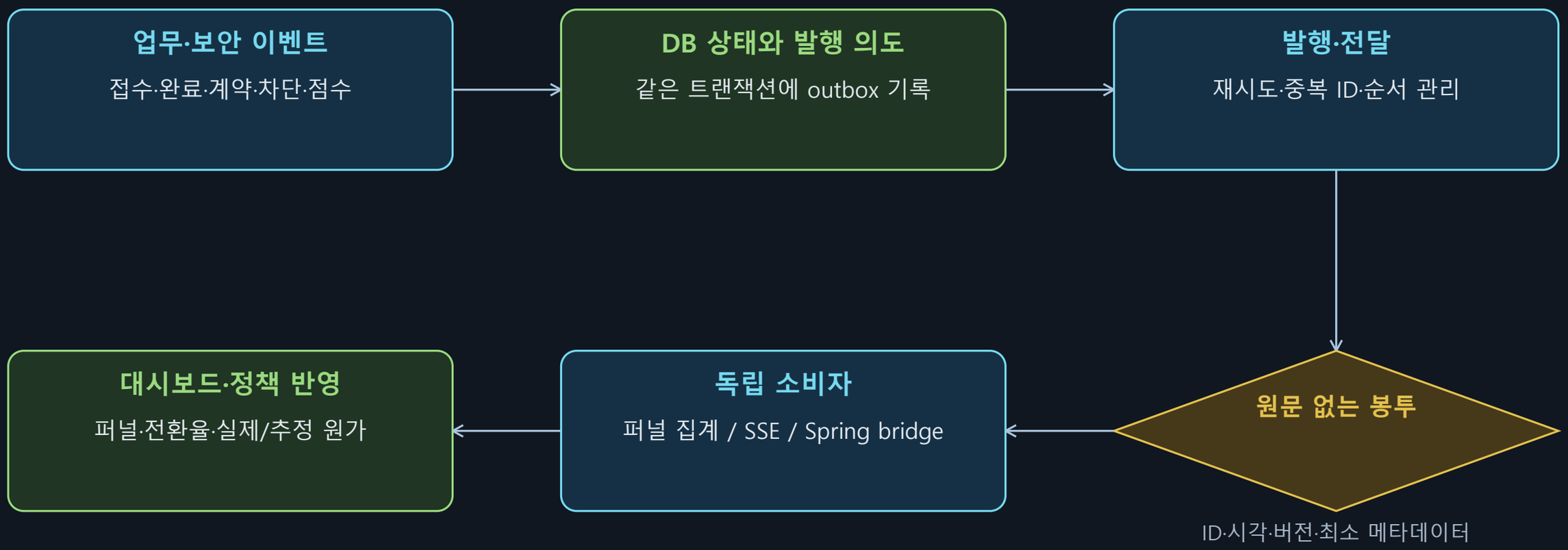
타임아웃·실패를 무조건 무료로 처리하지 않습니다. 실제 사용 USD와 고객 크레딧을 구분하고, 취소·부분 실행·사용량 미수신도 영속 원장에 상태로 남깁니다.

MIDDLE / EVENTS

08 · 도메인 이벤트와 퍼널

쉽게 말하면

서비스는 일어난 일을 이벤트로 남기고 통계·SSE·정책 소비자는 이를 읽습니다. 직접 호출 의존성을 줄이되 전달 실패와 중복도 처리해야 합니다.



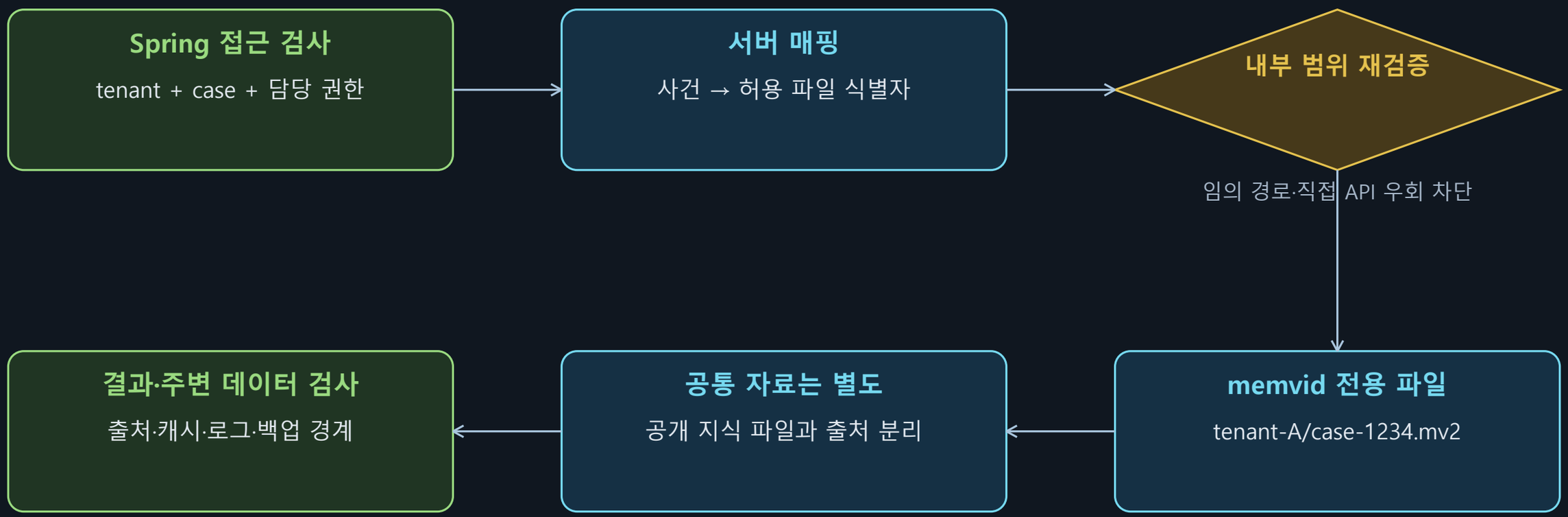
원문 이벤트: MarketingVisitRecorded, ConsultationRequested, ScreeningCompleted, BookingRequested/Confirmed, CaseRetained, IntrusionScoreCalculated, SecurityThresholdExceeded, SessionBlocked.

MIDDLE / MEMORY SCOPE

09 · 테넌트·사건별 memvid 격리

쉽게 말하면

로펌 분리만으로 사건별 권한까지 해결되지는 않습니다. 서버가 허용한 로펌·사건 조합을 전용 메모리 파일로 매핑하고 Python에서도 경계를 지킵니다.



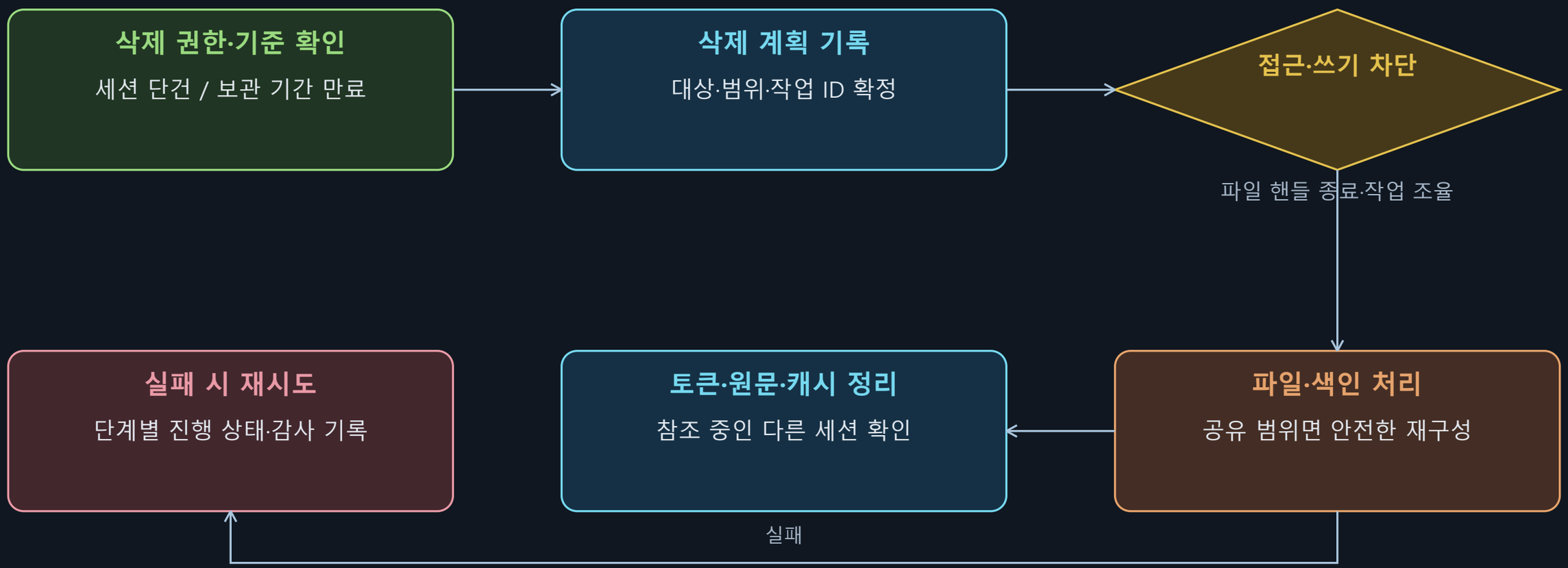
파일은 자동 암호화·완전 물리 격리를 뜻하지 않습니다. 키 관리·저장 암호화·ACL 집행을 별도 검증합니다. 관계형 저장소도 tenant 조건·허용 컬럼·불변 ID를 유지합니다.

MIDDLE / RETENTION

10-A · 개인정보 파기: DB와 파일의 차이

쉽게 말하면

DB 트랜잭션 하나가 memvid 파일과 백업 삭제까지 되돌려 주지는 않습니다. 삭제 계획을 먼저 확정하고, 파일·캐시·치환 표식의 정리를 재시도 가능한 작업으로 수행합니다.



같은 치환 표식이나 자료를 다른 세션이 쓰면 참조 관계를 확인합니다. 삭제 요청 후 조회·검색에 다시 나타나지 않는지 검증하고 장애 시에도 접근 차단을 유지합니다.

MIDDLE / DATA LIFECYCLE

10-B · 무엇을 지우고 무엇을 남기나

쉽게 말하면

사건 파일의 내용뿐 아니라 복호화 임시 파일·임베딩·스냅샷·백업·모델 전송 경로를 함께 봅니다. 논리 삭제를 저장 매체의 완전 삭제와 동일시하지 않습니다.

대상	memvid 개정판의 처리 원칙
세션·대화·보고서	정책상 삭제 범위와 업무상 보관 범위를 구분. 불필요한 원문 제거
사건 메모리	파일 단위 폐기 또는 허용 문서만 새 파일로 재구성 후 검증·교체
공유 치환 표식	다른 세션·사건 참조가 없을 때만 복원 원문 정리
예약·비용·보안 기록	필요 최소 기록과 근거를 유지하고 개인정보 필드를 최소화
백업·감사	삭제 일정·복원 시 재삭제·예외 정책을 기록. 감사에는 원문 대신 작업 ID·개수·상태

암호화 캡슐은 검색 전 복호화가 필요한 문서 흐름을 기준으로 설계합니다. 암호화된 저장본과 작업용 평문 모두 수명과 권한을 관리합니다.

BOTTOM / CONCURRENCY

11-A · 동시성: 기다림과 계산은 다르다

쉽게 말하면

가상 스레드와 async/await는 대기 중 자원 활용을 개선합니다. LLM 자체의 생성 속도나 로컬 임베딩의 CPU 계산을 자동으로 빠르게 만들지는 않습니다.

실행 방식	동작과 주의점	memvid 개정판의 선택
Java 21 가상 스레드	I/O 대기 중심의 동시 요청에 유리. 무한 동시 실행은 아님	활성화·HTTP/DB 연결 상한·타임아웃 확인
FastAPI 일반 def	요청 함수는 스레드 풀에서 실행되는 경로	동기 SDK·파일 작업을 무제한 늘리지 않음
async def + await	비동기 I/O를 기다리는 동안 다른 작업 가능	동기 검색·CPU 계산을 이벤트 루프에서 직접 막지 않음
로컬 임베딩·파일 쓰기	CPU·메모리·디스크·파일 잠금의 병목 존재	작업 큐·동시성 제한·사건별 쓰기 조율

원문 2026-10-27의 일반 def·약 40 토큰 제한·SQLite 단일 연결 설명은 당시 기록입니다. 이 문서는 현재 코드의 실행 모드를 재검증하지 않았습니다.

BOTTOM / PERFORMANCE

11-B · 속도와 효율: 먼저 측정할 것

쉽게 말하면

모델 호출이 길 수 있지만 고정된 밀리초 비율을 가정하지 않습니다. 인증·검색·임베딩·예약·생성·정산 시간을 각각 재고 전체 병목을 찾습니다.

병목	Spring의 역할	Python + memvid의 역할
대기·동시 요청	연결 상한·타임아웃·회로 차단	비동기 클라이언트 또는 제한된 실행 풀
같은 질문 반복	권한·정책·예산을 캐시로 우회하지 않음	사건·문서 버전·모델별 캐시 키와 폐기
모델 비용·지연	유료 단계 승인·상위 모델 전환 한도	근거만 선택, 출력량·재시도 제한
긴 작업	진행 상태·취소·재조회 계약	작업 큐·SSE·사용량 미확정 추적
장애 전파	AI 장애를 업무 기능과 분리	정상 실패 응답·추가 호출 중단·정산

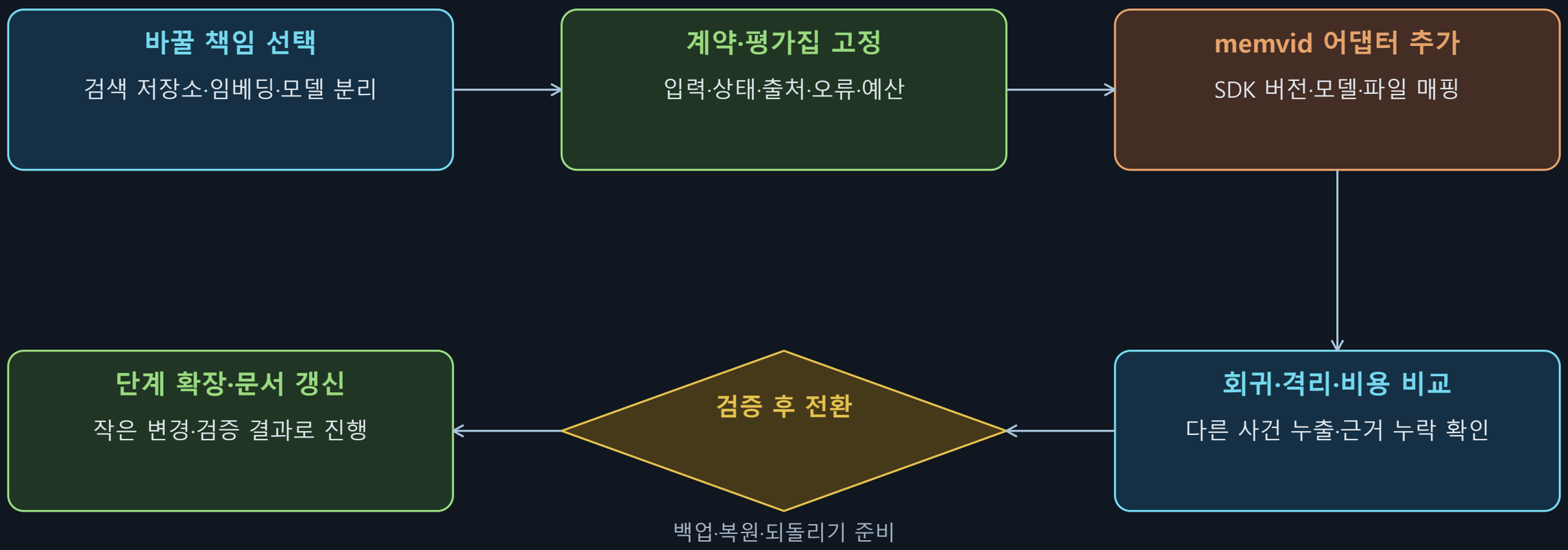
측정 지표: p50/p95 응답 시간, 검색 정확도, CPU·메모리, 대기열 길이, 재시도율, 요청당 비용. 조기 차단은 후속 비용을 줄이지만 이미 수행한 작업 비용을 0으로 만들지는 않습니다.

BOTTOM / REFACTORING

12-A · 계약을 고정하고 안쪽을 교체

쉽게 말하면

API의 바깥 모양과 권한·출처 계약을 먼저 테스트로 고정합니다. 그다음 검색 구현을 memvid 어댑터로 바꾸고 동일 질문의 근거와 비용을 비교합니다.



코어 SDK의 파일 생성·열기·검색은 어댑터 안에 모읍니다. 모델과 저장소를 동시에 바꾸면 효과를 분리하기 어려우므로 변경 범위를 나눠 평가합니다.

BOTTOM / INCREMENTAL CHANGE

12-B · 구현 사례와 점진 전환

쉽게 말하면

새 기능을 옆에 만들고 검증된 범위부터 옮깁니다. 그림의 이름은 제안 모듈이며 해당 코드가 이미 존재하거나 테스트를 통과했다는 뜻은 아닙니다.

변경 대상	작게 나누는 방법
의존성 조립	build_services 같은 조립 지점에서 메모리·모델·예산 클라이언트 주입
감사 로그	요청·응답 원문 대신 작업 ID·해시·최소 필드. 필요한 치환 매핑은 분리 보관
공급자 선택	데모→실제 모델의 자동 대체는 명시 정책·예산 승인 안에서만 수행
가드 위치 이전	일부 요청의 두 결과를 비교하고 오탐·누락 평가. 초기 검사는 Spring에 유지
메모리 이전	사건별 생성 → 문서 적재 → 재개방·검색 평가 → 승인 후 경로 전환

공개된 새 검색 경로로 민감 자료를 무단 복제하지 않습니다. 파일 포맷·임베딩 모델·출처 버전을 고정하고 이전 실패 시 기존 경로로 복구할 수 있게 합니다.

BOTTOM / DEVELOPMENT LANES

13 · 이 시스템을 만드는 방법

쉽게 말하면

독립된 구현은 분리해서 진행하고, 선행 데이터가 필요한 작업은 순서를 지킵니다. 이는 개발 방식 설명이며 이번 문서 변환에서 에이전트를 실행했다는 뜻은 아닙니다.



원문은 하루 테스트 116→300개 이상과 서로 다른 작업 폴더·브랜치 사례를 소개합니다. 과거 경험값이며 이번 개정판의 시험 성과가 아닙니다. 마스킹→저장→요약처럼 의존 작업은 순차 진행합니다.

BOTTOM / IMPLEMENTATION PLAN

모듈 지도: 이름보다 책임을 고정

쉽게 말하면

아래 경로는 새 구조를 설명하는 제안입니다. 원문 코드 경로를 새 이름으로 단순 치환해 실제 구현이라고 표시하지 않습니다.

제안 모듈	책임·확인할 조건
app/memory/memvid_store.py	사건 파일 생성·열기·검색·종료. 기존 파일 덮어쓰기 방지
app/memory/resolver.py	인증된 사건 범위→파일 매핑, 허용 루트 검사
app/embeddings/provider.py	로컬·외부 모델 선택, 문서/질문 모델 일치, 외부 전송 통제
app/screening/service.py	전처리→검색→예산 승인→생성→검증의 순서 조정
app/billing.py · spring_bridge.py	예산 원장 연동·중복 방지·사용량·보안 이벤트 보고
app/retention.py	파일·DB·캐시·참조 토큰의 삭제 계획·재시도·검증

필수 확인: 교차 사건 접근 차단, 저장 후 재개방, 파일 잠금, 모델 교체 시 재색인, 빈 검색 결과, 삭제 후 재등장, 동시 요청 예산, 타임아웃 후 정산.

APPENDIX / SOURCES

참고 근거와 개정 내역

쉽게 말하면

원본 PDF를 이미지로 확인해 13개 장을 재구성했습니다. 흰 배경·세로 도식을 다크 가로 도식으로 바꾸고 사건별 메모리·격리·삭제·비용 설명을 memvid 기준으로 개정했습니다.

자료	확인 범위
사용자 제공 PDF	Spring Boot _Fast API 하이브리드 구조 / 14쪽 중 내용 1-11쪽. 마지막 3쪽은 시각적으로 빈 페이지
memvid 공식 자료	프로젝트 · 권한 · 암호화 · 임베딩
실행 모델 공식 자료	FastAPI 동시성 · Java 21 가상 스레드
이번 검증	문서 재구성·가로 배치·한글 렌더링·벡터 도식·목차·용어 검사
검증하지 않은 것	SDK 실행·애플리케이션 코드 변경·서비스 배포·성능·보안 보장

외부 임베딩 모델은 선택 가능하지만 정확도를 자동 보장하지 않습니다. 같은 사건의 문서와 질문은 동일 모델 공간을 사용하고, 한국어 법률 평가집으로 근거 품질을 확인합니다.